

RTMP

Leonardo Duarte, Joel Díaz, Marc Mountoto, Beatriz Suarez, Adrià Trillo, Nicolas Guerra

Servicios de red ASIX2 2025

INTRODUCCIÓN

El *streaming* en vivo se ha convertido en una pieza clave de la comunicación digital, impulsando áreas como el entretenimiento, la educación y el marketing. En este contexto, el protocolo **RTMP** (*Real-Time Messaging Protocol*) sigue siendo relevante por su baja latencia y estabilidad, aunque enfrenta desafíos frente a alternativas modernas como HLS o WebRTC.

Este trabajo analiza la configuración práctica de RTMP usando herramientas como **OBS Studio** y **VLC**, abordando errores comunes y soluciones para transmisiones eficientes. Además, explora su papel actual en un entorno tecnológico en evolución, donde la optimización y seguridad son prioritarias.

El objetivo es ofrecer una guía clara para usuarios que buscan dominar el streaming con RTMP, combinando teoría y práctica para aplicaciones reales.

RTMP

Se trata de un protocolo de red que usa la capa de transporte de OSI, porque funciona sobre TCP. Por tanto, la mayoría de los servidores multimedia lo pueden procesar.

Era una tecnología propiedad de Adobe diseñada para la transmisión de alto rendimiento de audio, vídeo y datos. La mayoría de los codificadores pueden transmitir RTMP, y los grandes sitios de medios lo aceptan, pero la compatibilidad con los navegadores puede ser irregular.

¿CUÁLES SON LAS VENTAJAS Y DESVENTAJAS DEL RTMP VS OTROS PROTOCOLOS?

<i>Ventajas</i>	<i>Desventajas</i>
• Muy estable • Baja latencia • Alta compatibilidad • Compatibilidad con plataformas de streaming ○ Aún es el estándar para ingestión en plataformas como Twitch, YouTube Live y Facebook Live. ○ Establece conexiones rápidas con un handshake de 3 pasos (comparado con TLS en HTTP-based).	• Requiere conexión estable y persistente entre servidor y cliente • Al ser software propietario, hay poca flexibilidad para los usuarios avanzados. • Los navegadores modernos dejaron de soportar Flash Player (requiere reempaquetamiento a HLS/DASH para playback). • RTMP plano transmite datos sin cifrado (aunque puede usarse RTMPS sobre TLS para seguridad). • Problemas con firewalls y NAT • Usa puertos no estándar (1935 por defecto), lo que puede generar bloqueos en redes restrictivas. • Mantiene conexiones persistentes, lo que escala peor que protocolos basados en HTTP (como HLS). • En escenarios donde la pérdida de paquetes es aceptable (ej: videollamadas), WebRTC o SRT son más eficientes.

TABLA COMPARATIVA ENTRE RTMP VS OTROS PROTOCOLOS UTILIZADOS EN STREAMING

Características	RTMP (Real-Time Messaging Protocol)	HLS (HTTP Live Streaming)	DASH (Dynamic Adaptive Streaming)	WebRTC (Web Real-Time Communication)
Desarrollador	Adobe	Apple	MPEG	W3C / IETF
Latencia	1-5 segundos (baja)	10-30 segundos (alta)	10-30 segundos (alta)	< 1 segundo (ultra baja)
Protocolo de transporte	TCP	HTTP (TCP)	HTTP (TCP)	UDP / TCP (depende)
Compatibilidad con navegadores	No (requiere reempaquetamiento)	Sí (nativo en HTML5)	Sí (nativo en HTML5)	Sí (nativo en HTML5)
Adaptive Bitrate (ABR)	Sí (con configuración adicional)	Sí (nativo)	Sí (nativo)	Sí (pero menos común)
Seguridad	RTMPS (TLS) opcional	HTTPS (seguro por defecto)	HTTPS (seguro por defecto)	Encriptación obligatoria (DTLS-SRTP)

Uso principal	Ingestión de streams (envío)	Playback (entrega a usuarios)	Playback (entrega a usuarios)	Videollamadas / Streaming P2P
Escalabilidad	Media (conexiones persistentes)	Alta (HTTP estándar)	Alta (HTTP estándar)	Baja (óptimo para pocas conexiones)
Soporte en CDN	Limitado (en desuso para entrega)	Amplio	Amplio	Parcial (depende del proveedor)
Ventaja principal	Baja latencia para live streaming	Compatibilidad universal	Estándar abierto / Flexible	Ultra baja latencia (P2P)
Desventaja principal	No soporte nativo en navegadores	Alta latencia	Fragmentación de implementaciones	Consumo alto de ancho de banda

FUNCIONAMIENTO DEL RTMP

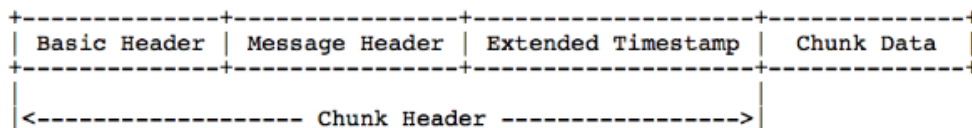
Transporta el contenido audiovisual en forma de datos entre codificador y la plataforma de streaming. Basado en la tecnología del Protocolo de Control de Transmisión (TCP) usando un puerto exclusivo (1935).

Real Time Messaging Protocol



CHUNKS

Es el sistema por el cual se multiplexa y empaquetan los paquetes para servicios de streaming.



FORMATO AAC Y H.264

La Codificación de vídeo avanzada (AVC), también conocida como H.264, es el estándar de compresión de vídeo que más se usa en la actualidad. El AVC/H.264 puede codificar vídeo de alta calidad con una velocidad de bits más baja que los estándares de compresión más antiguos.

AAC (del inglés Advanced Audio Coding) es un formato informático de señal digital audio basado en un algoritmo de compresión con pérdida, un proceso por el que se eliminan algunos de los datos de audio para poder obtener el mayor grado de compresión posible.

RECURSOS RECOMENDADOS PARA HACER MULTISTREAMING

- Hemos utilizado estos recursos visuales para realizarlo que recomendamos revisar para poder realizar una configuración en múltiples plataformas.

Tutorial para streamear en diferentes plataformas

[Live Streaming desde Cero - 1. Introduccion a la transmisión en vivo por internet.](#)

Video para utilizar twitch y obs, resumen interactivo

[CÓMO STREAMEAR EN TWITCH EN MENOS DE 5 MINUTOS](#)

Stream en múltiples plataformas al mismo instante,

[¡Transmite en Twitch y YouTube al Mismo Tiempo GRATIS! Tutorial Completo de Restream.io y OBS](#)

ALTERNATIVAS A RTMP

Lamentablemente el protocolo RTMP puede ser vulnerable a ataques de tipo Man In The Middle (MITM), para evitar dicho ataque se recomienda el uso del mismo protocolo sobre una conexión segura (TLS/SSL)

También podría usarse RTMPE, ***que se explicara en el siguiente punto.***

CONFIGURACION DE RTMP

1. Haz un resumen de la configuración y uso de RTMP (Real-Time Messaging Protocol Encrypted)

RTMP (Real-Time Messaging Protocol Encrypted) es una versión segura del protocolo RTMP, utilizado para la transmisión de audio, vídeo y datos en tiempo real.

1. Instalar un servidor RTMP: Se necesita un servidor como Nginx con el módulo RTMP o software especializado como Wowza Streaming Engine o Red5.
2. Generar claves de seguridad: Para habilitar la encriptación, es necesario configurar certificados SSL/TLS para asegurar la transmisión de datos entre el servidor y el cliente.

3. Configurar el servidor: En el archivo de configuración del servidor RTMP (por ejemplo, nginx.conf), se habilita el módulo de seguridad configurando puertos seguros y las claves SSL.
4. Configuración del cliente: En el software de transmisión (como OBS Studio), se configura la URL del servidor RTMP con la clave de transmisión y las credenciales necesarias.
5. Transmisión segura: Una vez configurado, el servidor y el cliente intercambian datos cifrados utilizando SSL/TLS, asegurando que la información transmitida esté protegida.

OBS, DOCKER Y DOCKER-COMPOSE PARA LA TRANSMISION DE UN STREAMING

Ahora explicaremos como poder realizar una transmisión usando **obs, docker, docker-compose, VLC y RTMP**.

1. Antes de realizar cualquier instalación lo recomendable es actualizar los repositorios de nuestra distribución GNU/Linux.

Debian / Debian-based

```
sudo apt update && sudo apt upgrade -y
```

```
apt install docker docker-compose obs-studio vlc
```

Arch / Arch-based

```
sudo pacman -Syu
```

```
sudo pacman -S docker docker compose obs-studio vlc
```

2. Una vez instalado es importante inicializar los servicios de docker

systemd

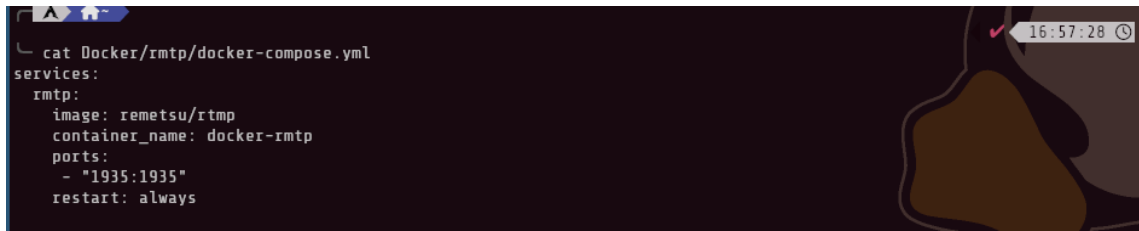
```
sudo systemctl start docker
```

SysVinit o Upstart

```
service docker start
```

3. Una vez instalado los requisitos procederemos a crear un compose

usando vim, nano o tu editor de preferencia.

A terminal window with a dark background. The prompt is a root symbol followed by a space. The command entered is 'cat Docker/rmtmp/docker-compose.yml'. The output shows a YAML configuration for a Docker service named 'rmtmp'. It specifies the image as 'remetsu/rtmp', the container name as 'docker-rmtmp', maps port 1935 of the container to port 1935 of the host, and sets the restart policy to 'always'.

```
cat Docker/rmtmp/docker-compose.yml
services:
  rmtmp:
    image: remetsu/rtmp
    container_name: docker-rmtmp
    ports:
      - "1935:1935"
    restart: always
```

vim docker-compose.yml para poner dentro lo siguiente:

1. services:

Este es el primer nivel en el archivo de configuración de Docker Compose. Define todos los contenedores que se van a crear y ejecutar. En este caso, se está declarando un servicio.

2. rmtmp:

Esta línea define el nombre del servicio que está siendo configurado. En este caso, el servicio se llama rmtmp.

3. image: remetsu/rtmp

Esta línea define la imagen de Docker que se va a usar para este servicio.

Vease: <https://hub.docker.com/r/remetsu/rtmp>

4. container_name: docker-rmtmp

Esta línea asigna un nombre al contenedor que se creará.

5. ports:

Este bloque define qué puertos de tu máquina local están mapeados a los puertos del contenedor. Esto es necesario para que puedas acceder al servicio que se está ejecutando en el contenedor desde fuera de él.

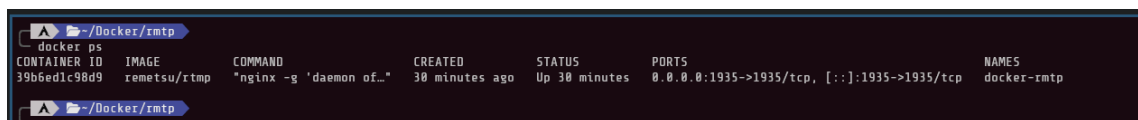
"1935:1935" significa que el puerto 1935 del contenedor será accesible en el puerto 1935 de tu máquina local. Este es el puerto

estándar utilizado para RTMP, por lo que esta configuración permite que el servidor RTMP sea accesible en tu máquina.

6. restart: always

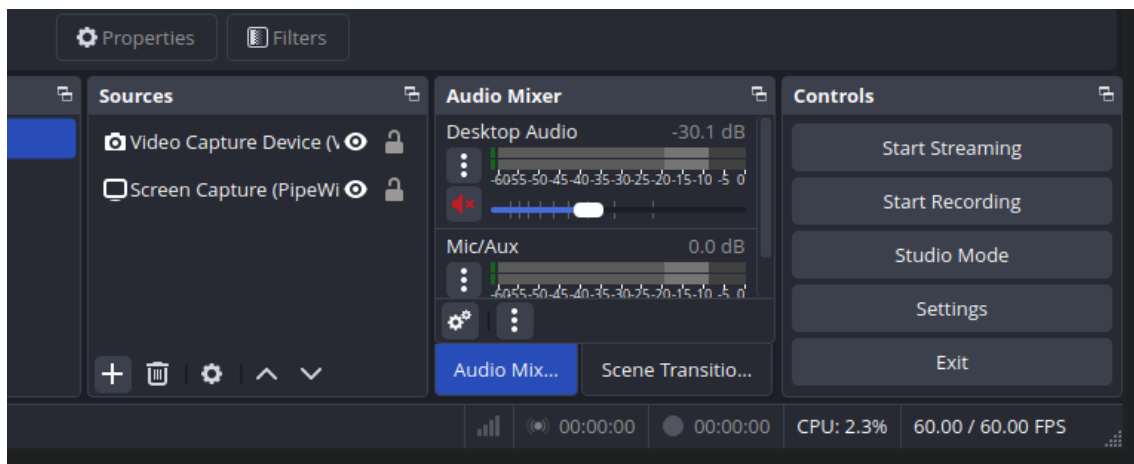
Esta línea configura la política de reinicio del contenedor en caso de que se detenga o falle.

Para ejecutarlo tenemos que situarnos en el directorio donde se encuentra el fichero que acabamos de crear (docker-compose.yml)



```
docker compose up -d
```

Ya que se encuentre corriendo nuestro contenedor procederemos a ejecutar OBS,



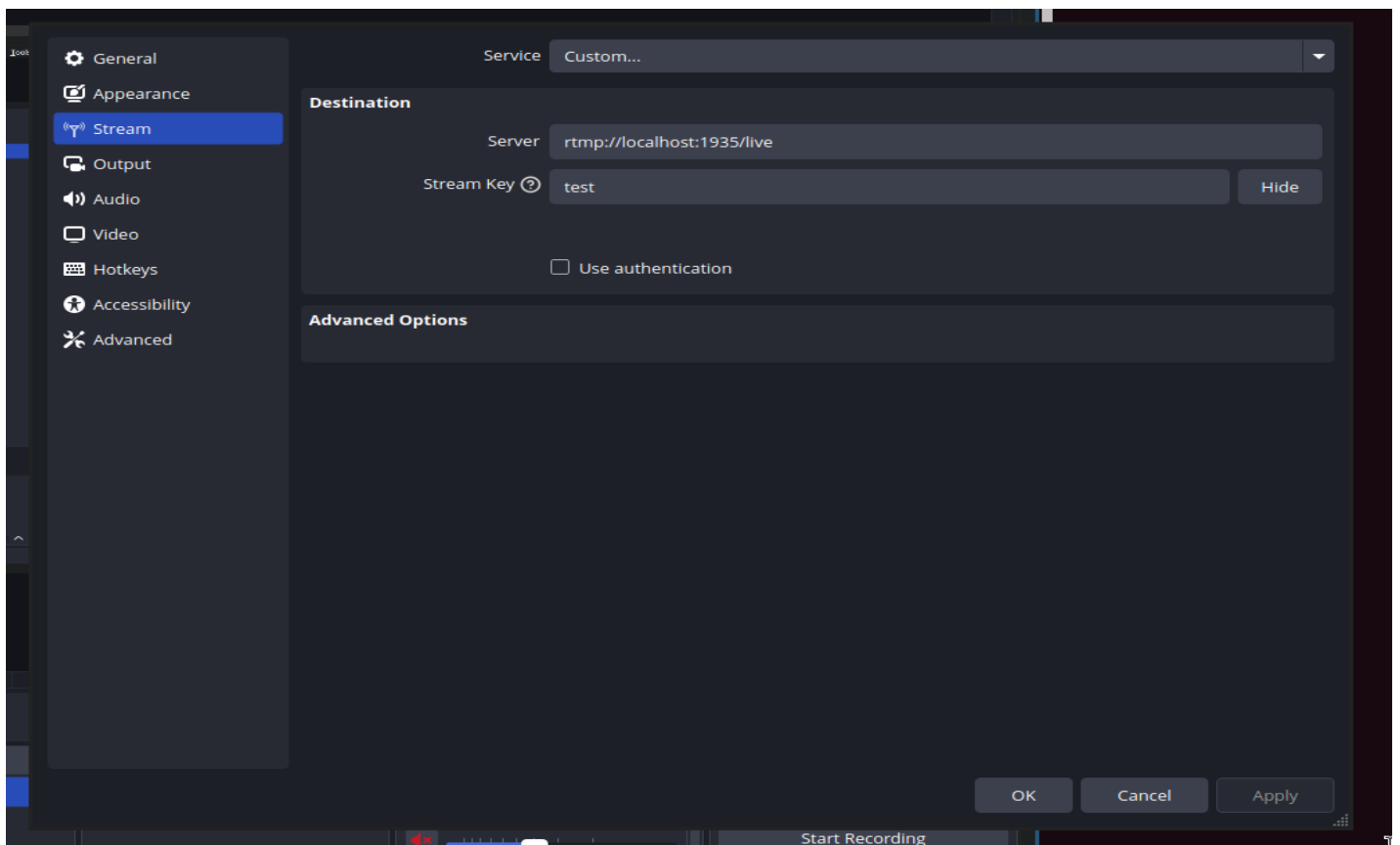
Ahora en el apartado de configuraciones pondremos lo siguiente

Service : Custom

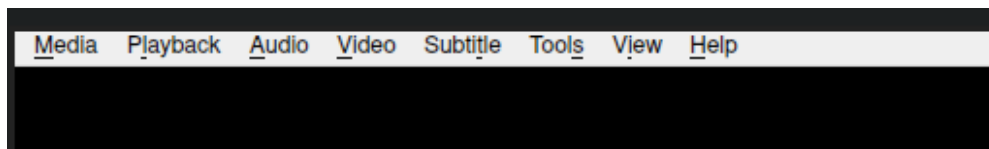
Server : localhost:1935/live

Stream Key : (lo que quieran, en mi caso test)

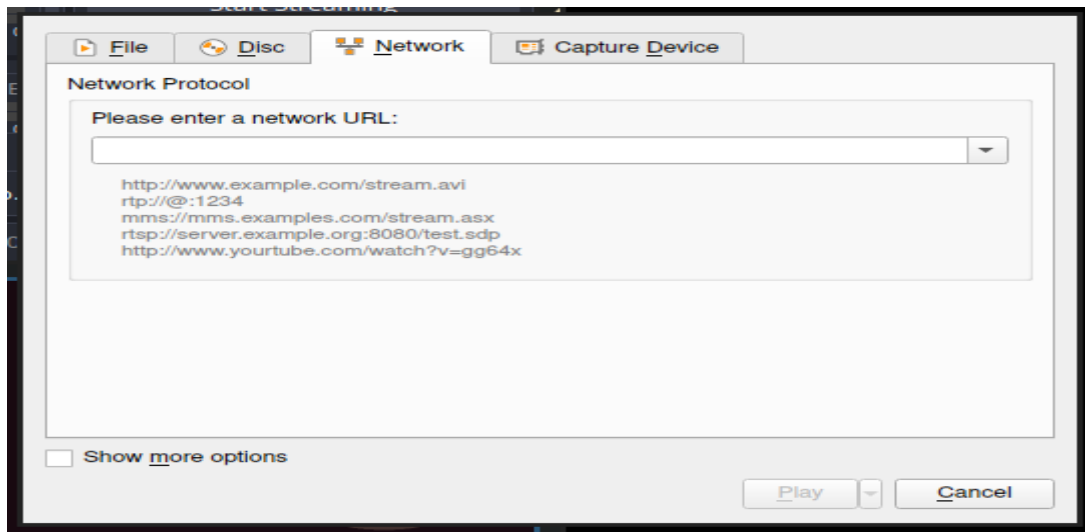
ok y luego apply



Presionamos en iniciar stream,
Ahora es cuando en vlc podemos visualizar la transmisión.



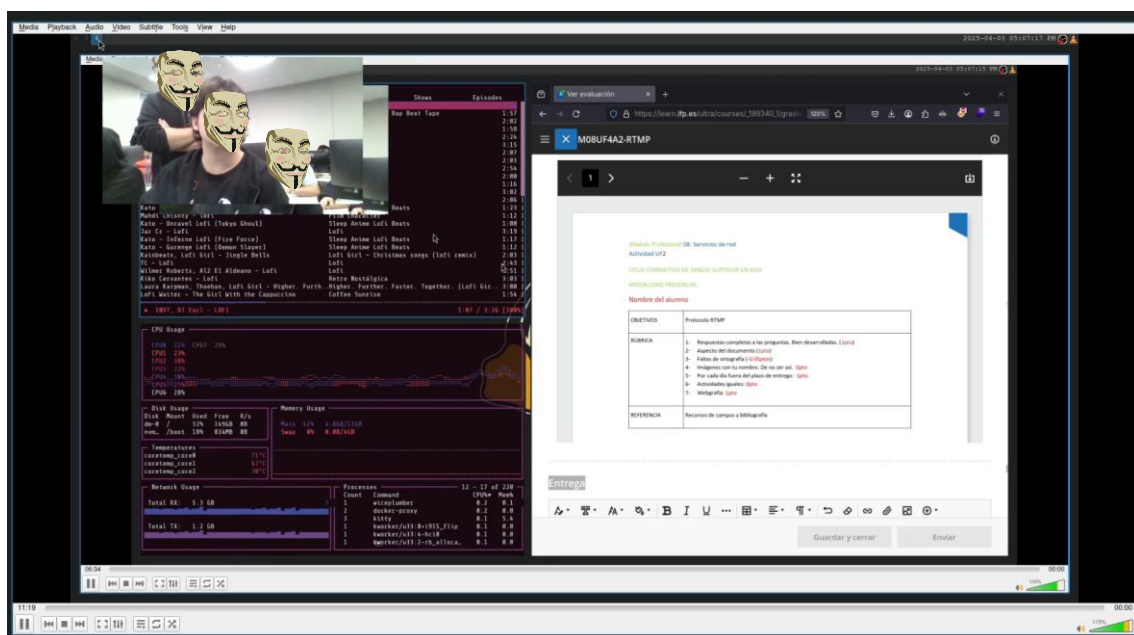
En el panel de arriba seleccionamos Media y Open Network stream o bien presionando el atajo de teclas ctrl + n.



Pondremos en la URL lo siguiente:

rtmp://localhost:1935/live/test

y ya estaría :D



2. CONFIGURACION DE RTMP

Primero instalamos el RTMP con el siguiente comando `<apt install libnginx-mod-rtmp>`

```
n_guerra@nguerra-VirtualBox:~$ sudo apt install libnginx-mod-rtmp
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
libnginx-mod-rtmp ya está en su versión más reciente (1.18.0-6ubuntu14.6).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
n_guerra@nguerra-VirtualBox:~$
```

Una vez instalado configuramos el NGINX.conf añadiendo lo siguiente lo siguiente

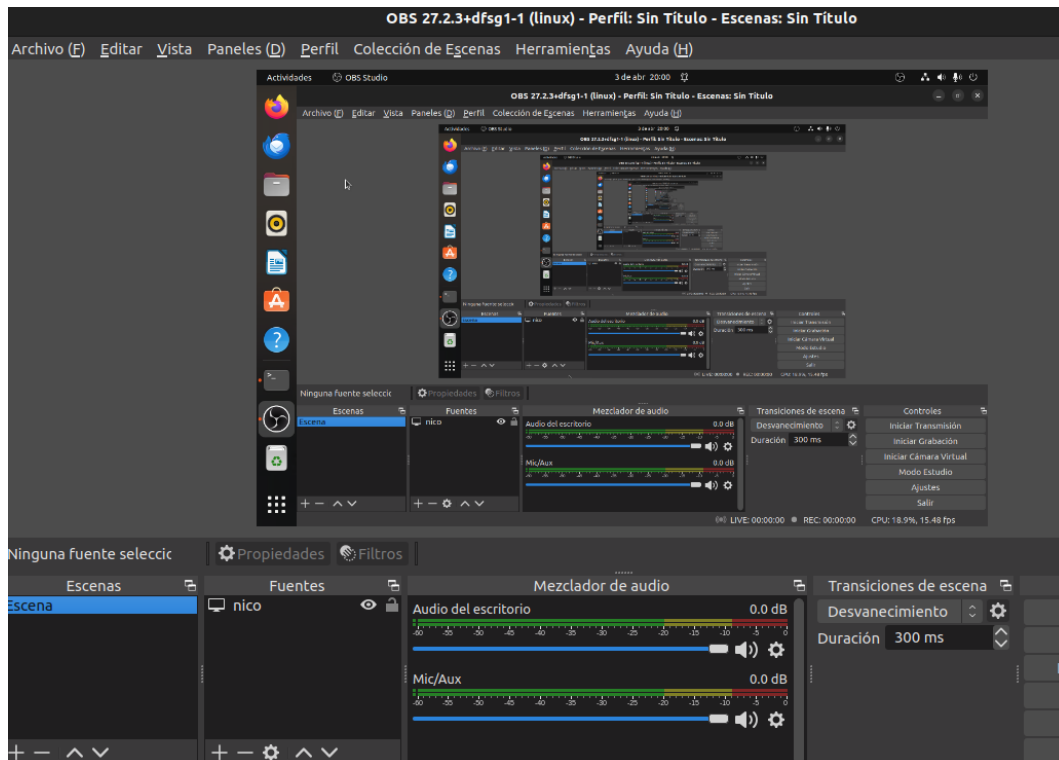
```
GNU nano 6.2 /etc/nginx/nginx.conf
#       listen      localhost:143;
#       protocol    imap;
#       proxy       on;
#     }
#}

rtmp {
    server {
        listen 1935;
        chunk_size 4096;
        allow publish 127.0.0.1;
        deny publish all;
        application live {
            live on;
            record off;
        }
    }
}
```

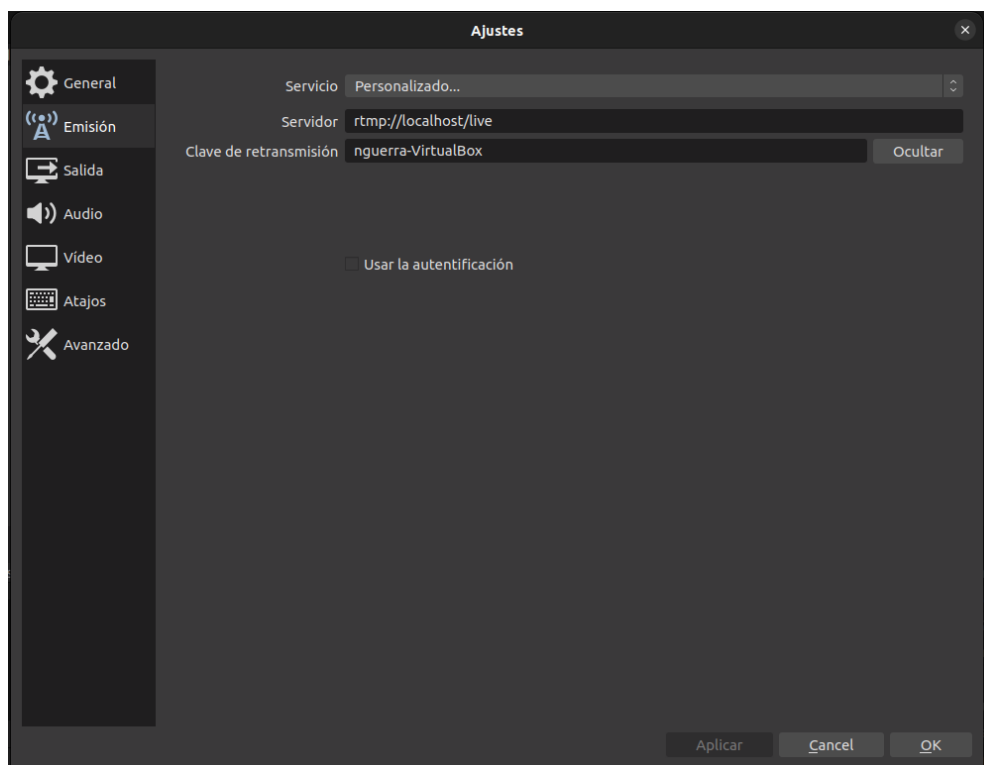
Después instalamos el OBS en nuestra máquina donde hicimos toda la configuración de nginx e RTMP

```
n_guerra@nguerra-VirtualBox:~$ sudo apt install obs-studio
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
obs-studio ya está en su versión más reciente (27.2.3+dfsg1-1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
n_guerra@nguerra-VirtualBox:~$
```

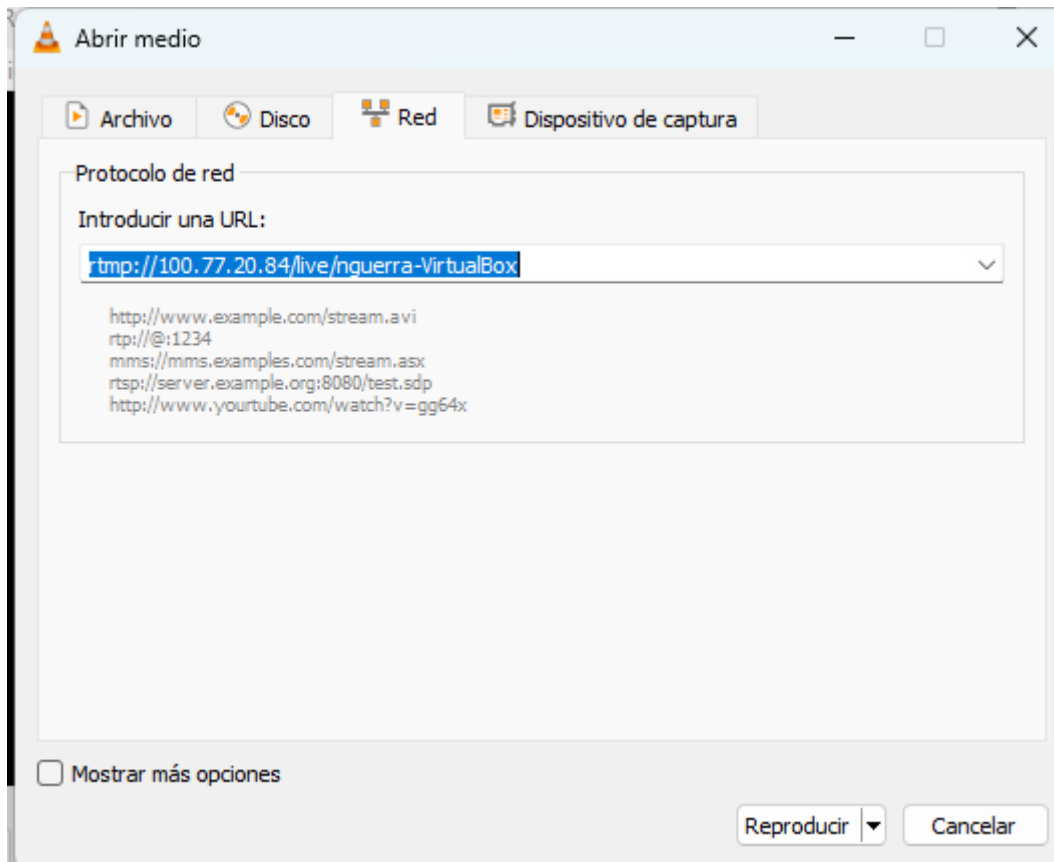
Abrimos el OBS (en caso que no te deje cierra sesión y abajo a la derecha cambia el Gnome) y configuramos lo siguiente. (para abrir el obs escribimos en la terminal obs)



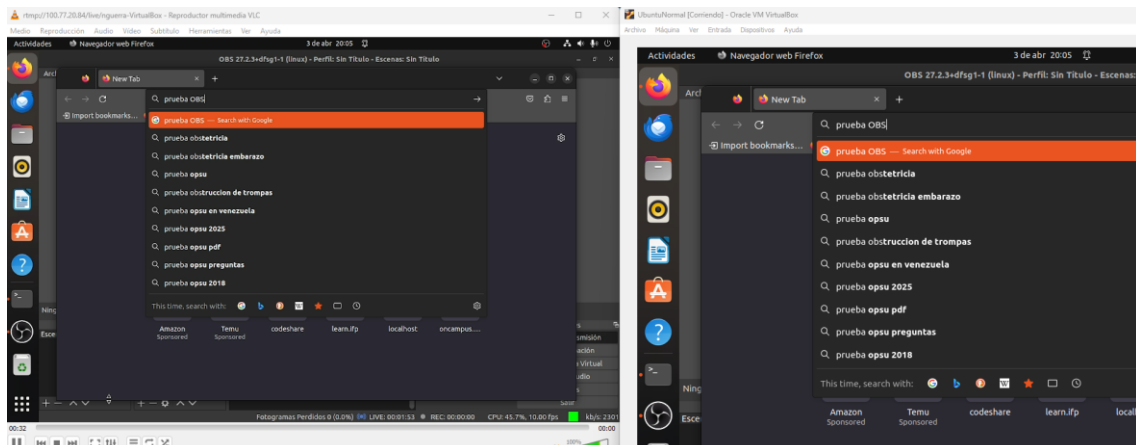
Procedemos a hacer la configuración de RTMP. Hay que irse a "Archivo" → "Ajustes" → "Emisión". Pondremos los ajustes en Personalizado y el servidor con lo siguiente (la clave es el host name de la maquina)



Iniciamos la transmisión y vamos al otro ordenador y pondremos lo siguiente en el VNC



Le damos a reproducir y veremos que se verá la pantalla que ponga la maquina donde esta el OBS



CONCLUSIÓN

El protocolo **RTMP** sigue siendo una tecnología relevante en el mundo del *streaming*, especialmente para transmisiones en vivo de baja latencia, aunque ha sido complementado (y en muchos casos reemplazado) por alternativas más modernas como **HLS**, **DASH** o **WebRTC**. Su uso principal hoy se centra en la **etapa de ingestión** (envío del stream a plataformas como Twitch o YouTube), mientras que la entrega a los espectadores se maneja con protocolos compatibles con HTML5.

A lo largo de este trabajo, se exploraron:

Las ventajas de RTMP: Baja latencia, estabilidad en redes fluctuantes y soporte para *adaptive bitrate*.

Sus limitaciones: Falta de soporte nativo en navegadores, dependencia de TCP y problemas con firewalls.

Configuraciones prácticas: Uso con **OBS Studio**, servidores como **NGINX-RTMP** y soluciones para multistreaming.

Para aplicaciones actuales, se recomienda:

- **Usar RTMP para la ingestión** (envío a plataformas).
- **Combinarlo con HLS/DASH para playback** (reproducción en navegadores).
- **Considerar WebRTC** si se requiere ultra baja latencia (ej: videollamadas interactivas).

En un panorama donde el *streaming* sigue creciendo en educación, entretenimiento y negocios, entender estas tecnologías permite optimizar la calidad, seguridad y alcance de las transmisiones. **RTMP no está muerto, pero su uso debe adaptarse al ecosistema actual de medios digitales.**

 Protocolos de Streaming**1. Wave.video (2023)**

Guía completa sobre RTMP: ¿Qué es y cuándo utilizarlo?

 <https://wave.video/blog/es/rtmp/>

2. Universidad Técnica Federico Santa María (2019)

Protocolo RTMP - Documento académico

 [https://profesores.elo.utfsm.cl/~agv/elo322/1s19/projects/reports/Protocolo RTMP.pdf](https://profesores.elo.utfsm.cl/~agv/elo322/1s19/projects/reports/Protocolo%20RTMP.pdf)

3. Adobe Developer

RTMPE: Real-Time Messaging Protocol Encrypted

 <https://www.adobe.com/devnet/rtmp.html> (Nota: Revisar link actualizado)

 Formatos de Codificación**4. Cloudflare (2023)**

¿Qué es H.264? Codificación de vídeo avanzada (AVC)

 <https://www.cloudflare.com/es-es/learning/video/what-is-h-264/>

5. Wikipedia

Advanced Audio Coding (AAC)

 https://es.wikipedia.org/wiki/Advanced_Audio_Coding

 Guías Prácticas**6. Escuela de Streaming**

Live Streaming desde Cero: Introducción

 <https://escueladestreaming.com/live-streaming-desde-cero/>

7. Twitch Creator Camp

Cómo transmitir en Twitch en 5 minutos

 <https://creatorcamp.twitch.tv/es-mx/basics/>

8. Restream.io Blog

Transmitir en Twitch y YouTube simultáneamente

 <https://restream.io/blog/es/transmitir-en-twitch-y-youtube/>